

IdleWorlds

Delegated Publisher Trust

Autonomous Signed Releases for the Fantasy Skin

ADDENDUM 1.2A | STANDING RELEASE AUTHORITY

Author	BustedCypher
Website	idleworldstoolkit.com
Repository	github.com/BustedCypher/IdleWorlds-3rd-Party-Reskin
Date	4 September 2026
Relationship	Follow-up to Full Community Skin Integration Counter-Proposal v1.2

Requested amendment: replace approval of every executable package hash with one revocable grant of standing authority to the Fantasy Skin publisher key. Valid signed releases may then update autonomously without a separate developer approval each time.

This addendum does not describe signatures as a substitute for trust. It explains how IdleWorlds can grant that trust deliberately, limit it to one named skin and publisher, observe every release, and withdraw it immediately if necessary.

1. The requested amendment

Counter-Proposal 1.2 separates publisher identity from developer approval. Under its primary method, a publisher signature proves who released a package, while an IdleWorlds-approved hash proves that the developer accepted those exact executable bytes. That model is cautious, but it still requires developer action for every JavaScript update.

This addendum asks IdleWorlds to trust the registered BustedCypher publisher key as the release authority for this one skin. A valid signature would become the authorization decision, not merely proof of authorship.

What changes

Question	Counter-Proposal 1.2	Proposed addendum
Who may publish?	Registered publisher	Registered publisher
What proves origin?	Publisher signature	Publisher signature
What authorizes execution?	Developer approval of each package hash	Standing trust in the publisher key
Can releases activate autonomously?	No, when executable bytes change	Yes, after automated verification
Can IdleWorlds stop future releases?	Yes	Yes, by revoking the key or skin ID
Can IdleWorlds roll back?	Yes	Yes, to a retained known-good version

What does not change

- The skin remains presentation-only and receives no gameplay API authority.
- Players must explicitly opt in and must be able to disable the skin.
- Unsigned, malformed, incompatible or revoked releases fail closed to the native interface.
- The developer retains control of the loader, eligibility, rollback and emergency disable mechanisms.
- A mutable main branch is never treated as a release and is never executed directly.

2. Autonomous signed release flow

The repository remains under BustedCypher's ownership and normal development may continue freely. IdleWorlds watches only the formal release channel. Ordinary commits, branches and pull requests have no production authority.



Release contents

File	Purpose
fantasy-skin-vX.Y.Z.zip	Versioned bundle containing the renderer, styles and assets
skin-manifest.json	Names the skin, publisher, version, compatible game range, package URL and package hash
skin-manifest.sig	Cryptographic signature over the exact manifest bytes
release notes	Human-readable description of visual, structural and executable changes

IdleWorlds verification sequence

- Confirm that the skin ID and publisher key are currently authorized.
- Verify the manifest signature before trusting any manifest field.
- Reject expired, replayed, malformed or incompatible manifests.
- Download the package with a short timeout and verify its complete cryptographic hash.
- Stage the package, retain the previous known-good release, then activate it.
- If startup fails, restore the previous version or the native IdleWorlds presentation.

No message to the developer is needed during this sequence. The standing publisher authorization is the approval, and every cryptographic and compatibility check is performed automatically.

3. The trust being requested

A signature proves that a release came from the holder of the private key and that its signed manifest was not altered. It does not prove that the JavaScript is safe, correct or free from mistakes.

Granting autonomous executable releases therefore means IdleWorlds is choosing to trust BustedCypher as a presentation-layer maintainer. In practical terms, this resembles granting a narrowly scoped maintainer permission: the authority is limited to one skin and can be revoked, but updates within that boundary no longer receive individual human approval.

This is more trust than the per-hash model in Counter-Proposal 1.2. It is also what makes independent maintenance possible. Without standing authority, every executable adjustment - including selector repair, responsive behavior, teardown handling or DOM compatibility - returns to the developer as a deployment request.

Safeguards that make the trust manageable

Control	Purpose	Owner
Dedicated release-signing key	Separates production authority from normal GitHub account access	BustedCypher
Protected release workflow	Only clean tagged builds that pass mandatory CI can become packages	Repository
Immutable artifacts and hashes	Prevents a published version from changing silently	Repository / host
Pinned dependencies and assets	Makes releases reproducible and limits supply-chain drift	BustedCypher
Compatibility declaration	Prevents known-incompatible game versions from loading	Both parties
Release audit log	Records skin ID, version, signer, hash, activation and rollback	IdleWorlds
Immediate key revocation	Stops all future releases from that publisher identity	IdleWorlds
Known-good rollback	Restores the last working package after failure or incident	IdleWorlds

These controls reduce operational and supply-chain risk. They do not remove the need for trust; they make that trust explicit, observable, limited and reversible.

4. Authority boundaries and incident control

Autonomous publishing does not require IdleWorlds to surrender control of its application. The trust grant can be narrowly bound to a publisher identity, one skin ID, an approved package location and a defined presentation-only capability set.

BustedCypher controls

- Repository ownership, branches, commits and development priorities.
- When a tested version becomes a formal signed release.
- The skin's renderer, artwork, fonts, layouts, responsive behavior and presentation lifecycle.
- Release notes and compatibility declarations supplied with each version.

IdleWorlds controls

- The first-party loader and all access it exposes to a skin.
- Which accounts or players may opt in and how consent is stored.
- Maximum package size, allowed origins, timeouts and compatibility policy.
- The active publisher key, revocation list and emergency global kill switch.
- Retention and activation of known-good versions for rollback.

Incident sequence

Event	Automatic or developer response
Invalid signature or altered manifest	Reject before downloading or executing the package
Package hash mismatch	Reject the package and retain the current known-good release
Startup or health check failure	Roll back automatically and record the failure
Compromised signing key	Revoke the publisher key; reject every later release signed by it
Serious skin regression	Disable the current version or the entire skin without changing repository ownership

The developer's ultimate authority is the ability to refuse, revoke, roll back or disable. BustedCypher's operating independence is the ability to publish routine signed releases without requesting a new approval each time.

5. Proposed operating agreement and decision

One-time onboarding

- IdleWorlds and BustedCypher agree the package structure, manifest schema and signing algorithm.
- BustedCypher supplies the public release key; the private key remains solely under publisher control.
- IdleWorlds registers that public key against the Fantasy Skin's immutable skin ID.
- Both parties validate one release end to end, including rejection, rollback and revocation tests.
- After the pilot succeeds, IdleWorlds enables standing release authority for the publisher key.

Normal operation

- BustedCypher develops and commits without seeking IdleWorlds approval.
- Only an intentional, versioned and cryptographically signed release can reach the loader.
- IdleWorlds performs automated signature, hash, compatibility and health checks.
- A valid release activates for opted-in users without a developer review step.
- IdleWorlds may investigate, roll back or revoke authority at any time if a material problem appears.

Decision requested: Is IdleWorlds willing to replace per-release executable hash approval for the Fantasy Skin with a revocable standing authorization for releases signed by BustedCypher's registered publisher key?

Suggested acceptance conditions

Condition	Acceptance test
Authenticity	A valid release loads; any unsigned or differently signed release is rejected
Integrity	Changing one package byte causes hash verification to fail
Autonomy	A second valid signed release activates without manual developer approval
Resilience	Network and startup failures preserve the current release or native presentation
Revocation	Revoking the publisher key blocks a later correctly signed release
Recovery	IdleWorlds can restore a known-good version independently of the repository

Closing position

This amendment offers a clear exchange: IdleWorlds accepts a bounded trust relationship with one named maintainer, and the Fantasy Skin gains the autonomy required to remain compatible and actively maintained. The system does not pretend trust can be eliminated; it gives both parties a precise way to grant, use, observe and revoke it.